
# C#-Reference and Value Types

# Objective



- Value Type and Reference Type
- Value Types
- Structures
- Enum
- Reference Types
- out and ref
- nullable types
- nullable reference types
- ? and ??
- boxing and unboxing

# Value Type and Reference Type

- In C#, these data types are categorized based on how they store their value in the memory. C# includes the following categories of data types:

1. **Value type**
2. **Reference type**
3. **Pointer type**

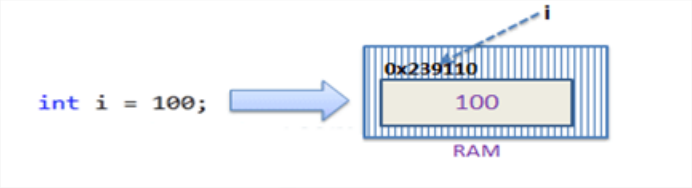
# Value type

- A data type is a value type if it holds a data value within its own memory space. It means the variables of these data types directly contain values.
- Value types stored on the stack and are passed by copying.
- This means that when a value type variable is created, the memory for it is allocated on the stack. The stack is a region of memory that is used to store temporary data, such as function arguments and local variables. Because value types are stored on the stack, they are typically accessed more quickly than reference types, which are stored on the heap.
- Value types in C# include simple types such as integers, floating-point numbers, and Booleans, as well as enumerations and structures. In contrast, reference types, such as objects and arrays, are stored on the heap and are accessed through references.
- All the value types derive from ***System.ValueType***.

- Example

  consider integer variable int i = 100;

- The system stores 100 in the memory space allocated for the variable i. The following image illustrates how 100 is stored at some hypothetical location in the memory (0x239110) for 'i':



- **Value types include the following**:
- All numeric data types
- Boolean , Char ,and Date
- All structures, even if their members are reference types .
- Enumerations, since their underlying type is always SByte , Short , Integer , Long , Byte , UShort , UInteger , or ULong
- Every structure is a value type, even if it contains reference type members. For this reason, value types such as Char and Integer are implemented by .NET Framework structures.
- **Passing Value Type Variables**
- When you pass a value-type variable from one method to another, the system creates a separate copy of a variable in another method. If value got changed in the one method, it wouldn't affect the variable in another method.

# Structures

- ***Structure*** is a value type and a collection of variables of different data types under a single unit.
- It is almost similar to a class because both are user-defined data types and both hold a bunch of different data types

**Defining Structure:**

- In C#, structure is defined using ***struct*** keyword.
- Using struct keyword one can define the structure consisting of different data types in it.
- A structure can also contain constructors, constants, fields, methods, properties, indexers and events etc.

- **Syntax:**

```
Access_Modifier struct structure_name
{
   // Fields
  // Parameterized constructor
  // Constants
  // Properties
  // Indexers
  // Events
  // Methods etc.

}
```

```
// Defining structure
public struct Person
{
    // Declaring different data types
    public string Name;
    public int Age;
    public int Weight;

}
```

```
// Main Method
static void Main(string[] args)
{
    // Declare P1 of type Person
    Person P1;

    // P1's data
    P1.Name = "Keshav Gupta";
    P1.Age = 21;
    P1.Weight = 80;

    // Displaying the values
    Console.WriteLine("Data Stored in P1 is " + P1.Name + ", age is " +
                      P1.Age + " and weight is " + P1.Weight);

}
```

# Difference Between Structures and Class :

OBJECT TECHNOLOGIES

| Class | Structure |
|---|---|
| Classes are of reference types. | Structs are of value types. |
| All the reference types are allocated on heap memory. | All the value types are allocated on stack memory. |
| Allocation of large reference type is cheaper than allocation of large value type. | Allocation and de-allocation is cheaper in value type as compare to reference type. |
| Class has limitless features. | Struct has limited features. |
| Class is generally used in large programs. | Struct are used in small programs. |
| Classes can contain constructor or destructor. | Structure does not contain parameter less constructor or destructor, but can contain Parameterized constructor or static constructor. |
| Classes used new keyword for creating instances. | Struct can create an instance, with or without new keyword. |
| A Class can inherit from another class. | A Struct is not allowed to inherit from another struct or class. |
| The data member of a class can be protected. | The data member of struct can't be protected. |
| Function member of the class can be virtual or abstract. | Function member of the struct cannot be virtual or abstract. |
| Two variable of class can contain the reference of the same object and any operation on one variable can affect another variable. | Each variable in struct contains its own copy of data(except in ref and out parameter variable) and any operation on one variable can not effect another variable. |

# enum

- Enumeration (or enum) is a value data type in C#.
- It is mainly used to assign the names or string values to integral constants, that make a program easy to read and maintain.
- For example, the 4 suits in a deck of playing cards may be 4 enumerators named Club, Diamond, Heart, and Spade, belonging to an enumerated type named Suit.
- Enumeration is declared using ***enum*** keyword directly inside a namespace, class, or structure.

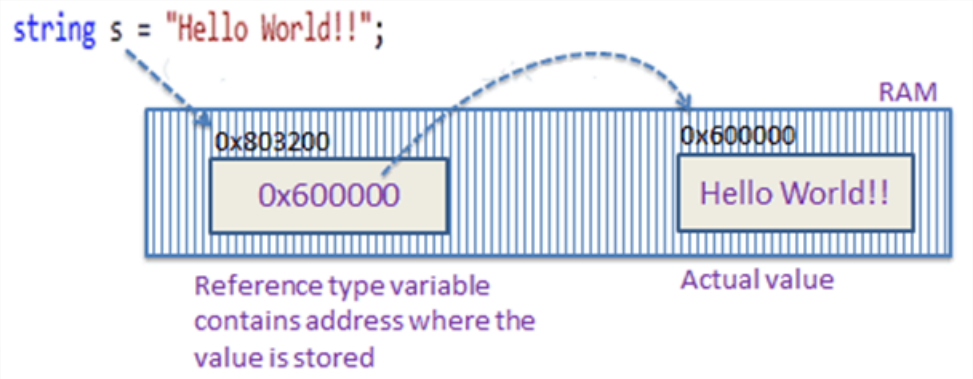
**Syntax:**

```
enum Enum_variable
{
    string_1...;
    string_2...;
    .
    .
}
```

```
// making an enumerator 'month'
enum month
{
    // following are the data members
    jan,
    feb,
    mar,
    apr,
    may

}

class Program
{
    // Main Method
    static void Main(string[] args)
    {

        // getting the integer values of data members..
        Console.WriteLine("The value of jan in month " +
                          "enum is " + (int)month.jan);
        Console.WriteLine("The value of feb in month " +
                          "enum is " + (int)month.feb);
        Console.WriteLine("The value of mar in month " +
                          "enum is " + (int)month.mar);
        Console.WriteLine("The value of apr in month " +
                          "enum is " + (int)month.apr);
        Console.WriteLine("The value of may in month " +
```

# Reference Types

- reference type doesn't store its value directly. Instead, it stores the address where the value is being stored. In other words, a reference type contains a pointer to another memory location that holds the data.
- For example, consider the following string variable:
  string s = "Hello World!!";
- The following image shows how the system allocates the memory for the above string variable.



- **The followings are reference type data types**:

1. String
2. Arrays (even if their elements are value types)
3. Class
4. Delegate

- The default value of a reference type variable is null when they are not initialized. Null means not referring to any object.
- The reference types are stored on heap. When assigning one reference variable to another reference variable, only the reference is copied. The actual value remains the same.

# out and ref

- Ref and out keywords in C# are used to pass arguments within a method or function. Both indicate that an argument/parameter is passed by reference.
- By default parameters are passed to a method by value. By using these keywords (ref and out) we can pass a parameter by reference.
- **Ref Keyword**
- The ref keyword passes arguments by reference.
- It means any changes made to this argument in the method will be reflected in that variable when control returns to the calling method.
- **Out Keyword**
- The out keyword passes arguments by reference. This is very similar to the ref keyword.

# Ref Example

OBJECT TECHNOLOGIES

```
{
    static void Main(string[] args)
    {
        int Addition = 0;
        int Multiplication = 0;
        int Subtraction = 0;
        int Division = 0;
        Math(200, 100, ref Addition, ref Multiplication, ref Subtraction, ref Division);

        Console.WriteLine($"Addition: {Addition}");
        Console.WriteLine($"Multiplication: {Multiplication}");
        Console.WriteLine($"Subtraction: {Subtraction}");
        Console.WriteLine($"Division: {Division}");

        Console.ReadKey();
    }

    public static void Math(int number1, int number2, ref int Addition, ref int Multiplication,
        ref int Subtraction, ref int Division)
    {
        Addition = number1 + number2;
        Multiplication = number1 * number2;
        Subtraction = number1 - number2;
        Division = number1 / number2;
    }
}
```

# Out Example

OBJECT TECHNOLOGIES

```
static void Main(string[] args)
{
    int Addition = 0;
    int Multiplication = 0;
    int Subtraction = 0;
    int Division = 0;
    Math(200, 100, out Addition, out Multiplication, out Subtraction, out Division);

    Console.WriteLine($"Addition: {Addition}");
    Console.WriteLine($"Multiplication: {Multiplication}");
    Console.WriteLine($"Subtraction: {Subtraction}");
    Console.WriteLine($"Division: {Division}");

    Console.ReadKey();
}

public static void Math(int number1, int number2, out int Addition,
    out int Multiplication, out int Subtraction, out int Division)
{
    Addition = number1 + number2;
    Multiplication = number1 * number2;
    Subtraction = number1 - number2;
    Division = number1 / number2;
}
```

| Ref | Out |
|---|---|
| The parameter or argument must be initialized first before it is passed to ref. | It is not compulsory to initialize a parameter or argument before it is passed to an out. |
| It is not required to assign or initialize the value of a parameter (which is passed by ref) before returning to the calling method. | A called method is required to assign or initialize a value of a parameter (which is passed to an out) before returning to the calling method. |
| Passing a parameter value by Ref is useful when the called method is also needed to modify the pass parameter. | Declaring a parameter to an out method is useful when multiple values need to be returned from a function or method. |
| It is not compulsory to initialize a parameter value before using it in a calling method. | A parameter value must be initialized within the calling method before its use. |
| When we use REF, data can be passed bi-directionally. | When we use OUT data is passed only in a unidirectional way (from the called method to the caller method). |
| **Both ref and out are treated differently at run time and they are treated the same at compile time.** | |
| **Properties are not variables, therefore it cannot be passed as an out or ref parameter.** | |

# nullable types

- In C#, the compiler does not allow you to assign a null value to a variable. So, C# 2.0 provides a special feature to assign a null value to a variable that is known as the Nullable type.
- The Nullable type allows you to assign a null value to a variable. Nullable types introduced in C#2.0 can only work with Value Type, not with Reference Type.
- The nullable types for Reference Type is introduced later in C# 8.0 in 2019 so that we can explicitly define if a reference type can or can not hold a null value.
- The Nullable type is an instance of System.Nullable<T> struct. Here T is a type which contains non-nullable value types like integer type, floating-point type, a boolean type, etc. For example, in nullable of integer type you can store values from -2147483648 to 2147483647, or null value.

- **Syntax**:

**Nullable<data_type> variable_name = null;**

Or

**datatype? variable_name = null;**

**Example:**

```
// this will give compile time error
int j = null;

// Valid declaration
Nullable<int> j = null;

// Valid declaration
int? j = null;
```

# Characteristics of Nullable types

- With the help of nullable type you can assign a null value to a variable without creating nullable type based on the reference type.
- In Nullable types, you can also assign values to nullable type.
- You can use **Nullable.HasValue** and **Nullable.Value** to check the value. If the object assigned with a value, then it will return "True" and if the object is assigned to null, then it will return "False". If the object is not assigned with any value then it will give compile-time error.
- You can also use == and ! operators with nullable type.
- You can also use **GetValueOrDefault(T)** method to get the assigned value or the provided default value, if the value of nullable type is null.
- You can also use **null-coalescing operator(??)** to assign a value to the underlying type originate from the value of the nullable type.
- Nullable types do not support var type. If you use Nullable with var, then the compiler will give you a compile-time error.

# Advantage of Nullable Types:

- The main use of nullable type is in database applications. Suppose, in a table a column required null values, then you can use nullable type to enter null values.
- Nullable type is also useful to represent undefined value.
- You can also use Nullable type instead of a reference type to store a null value.

```
// a is nullable type
// and contains null value
int? a = null;

// b is nullable type int
// and behave as a normal int
int? b = 2345;
```

# Example

```
int? a = null;
int? b = 2345;
// using the method
Console.WriteLine(b.GetValueOrDefault());
Console.WriteLine(a.HasValue);
Console.WriteLine(a);
```

# ? and ??

- The null-coalescing operator **??** returns the value of its left-hand operand if it isn't null; otherwise, it evaluates the right-hand operand and returns its result.
- The **??** operator doesn't evaluate its right-hand operand if the left-hand operand evaluates to non-null.
- The null-coalescing assignment operator **??=** assigns the value of its right-hand operand to its left-hand operand only if the left-hand operand evaluates to null.
- The **??=** operator doesn't evaluate its right-hand operand if the left-hand operand evaluates to non-null.

```
int? a = null;
int b = a ?? -1;
Console.WriteLine(b);  // output: -1
```

```
int? a = null;
int b = a ??= -1;
Console.WriteLine(b);  // output: -1
Console.WriteLine(a);// output: -1
```

# nullable reference types

- Since the introduction of generics in C# 2, value types can be declared nullable or non-nullable:

```
int nonNullable = null; // compiler error
int? nullable = null;
```

- In C# 8, nullable reference types use the same syntax to give the option of declaring reference types as nullable (i.e. allowing a null value) or non-nullable (not allowing a null value):

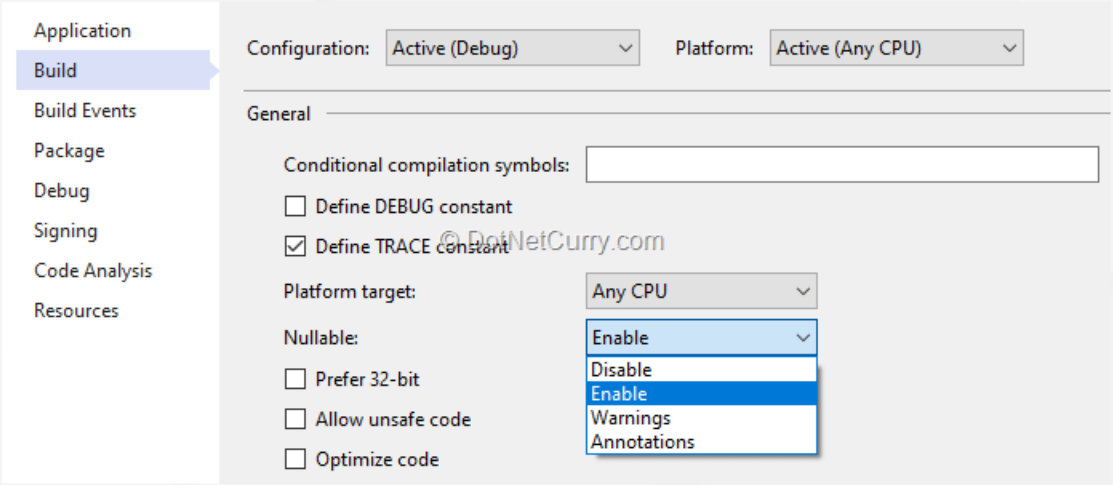
```
string nonNullable = null; // compiler warning
string? nullable = null;
```

- Because of the language history, the decision to use the same syntax for value types and reference types changes the behavior of the language for reference types. Before C# 8, all reference types were nullable. They were however declared without a question mark:

```
// before C# 8
string nullableString;
// since C# 8
string nonNullableString;
```

- To avoid such a breaking change in a new version of the language, the nullable reference types is the only feature of C# 8 that isn't enabled by default. An explicit opt-in is required for each project. The following property must be added to the project file for that: **<Nullable>enable</Nullable>**

- In recent versions of Visual Studio 2019, the option is also available on the Build page of the Project Properties window:



# Boxing And Unboxing

- Boxing and unboxing are important concepts in C#.
- The C# Type System contains three data types: Value Types (int, char, etc), Reference Types (object) and Pointer Types.
- Basically, Boxing converts a Value Type variable into a Reference Type variable, and Unboxing achieves the vice-versa.
- Boxing and Unboxing enable a unified view of the type system in which a value of any type can be treated as an object.

# Boxing In C#

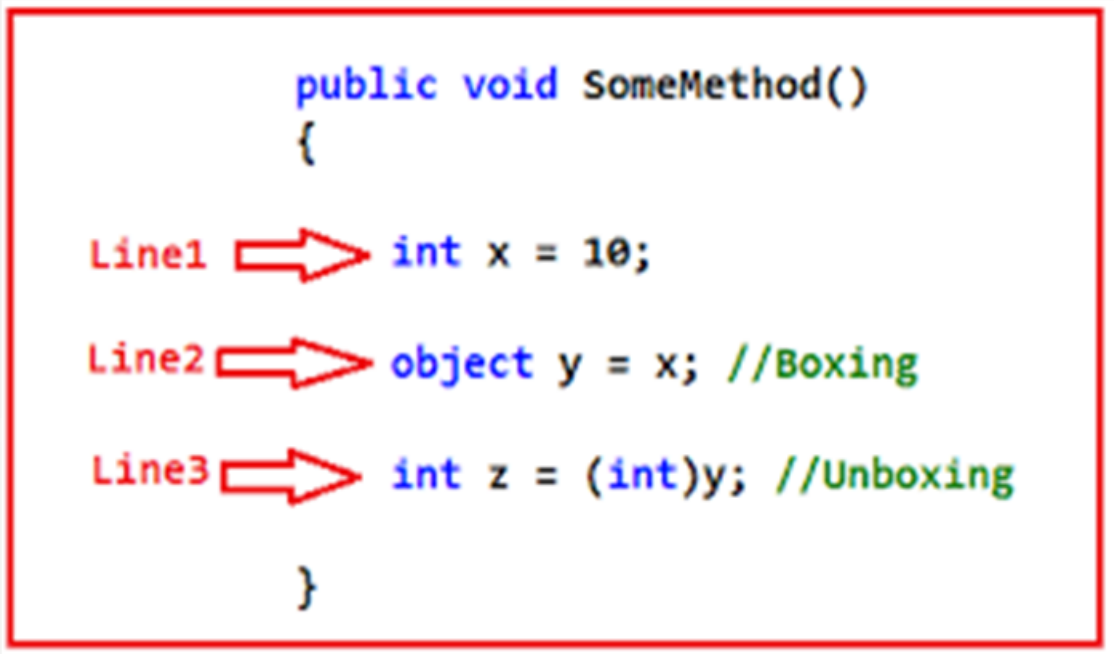
- The process of converting a Value Type variable (char, int etc.) to a Reference Type variable (object) is called Boxing.
- Boxing is an implicit conversion process in which object type (super type) is used.
- Value Type variables are always stored in Stack memory, while Reference Type variables are stored in Heap memory.
- **Example** :

```
int num = 23; // 23 will assigned to num
Object Obj = num; // Boxing
```

# Unboxing In C#

- The process of converting a Reference Type variable into a Value Type variable is known as Unboxing.
- It is an explicit conversion process.
- **Example** :

```
int num = 23;        // value type is int and assigned value 23
Object Obj = num;    // Boxing
int i = (int)Obj;    // Unboxing
```


public void SomeMethod()
{
Line1
int x = 10;
Line2
object y = x; //Boxing
Line3
int z = (int)y; //Unboxing
}

# Summary

- Value Type and Reference Type
- Value Types
- Structures
- Enum
- Reference Types
- out and ref
- nullable types
- nullable reference types
- ? and ??
- boxing and unboxing


# Thank You